

(version 3) HowToSAPC 2026

Introduction

Ce manuel recouvre l'ensemble des pistes conseillées (et validées) pour mener à bien votre projet.

Les codes partiels (.h et .cpp) éventuellement fournis sont tous fonctionnels. Ils vous appartient de les comprendre et de les coder, de les améliorer durant chaque phase du projet. Pensez donc aussi à mettre des commits (publish réguliers commentés) sur branches de votre git !

Ces codes sont obtenus par compilation au travers du *Makefile* fourni.

Compilation

Selon la cible (PC, ou Pi1, Pi3) vous devrez adapter vos programmes binaires (PAS les sources!), vu que nous sommes potentiellement sur une cross-compilation (processeur cible a l'architecture matérielle (des registres, chipset CISC(pc intel) ou RISC (pi)) différente de celle ou vous compilez. Le plus simple est de compiler "nativement" sur la cible (le raspberry).

Les commandes possibles selon :

- Création des 4 librairies : rest, mcp, ina bdd, pour respectivement le serveur Rest, le code du MCP3008, l'INA226, et la base de données : `make librest`, `make libmcp`, `make libina`, `make libdd`
- Effacement de ces librairies + binaire global : *make cleanall*
- Création des binaires globaux (un juste pour test capteur et l'autre avec Rest en plus) :
capteurs seuls : *make all*

Le serveur REST

1. Installation librairie json:nlohmann

```
sudo apt update  
sudo apt install nlohmann-json3-dev
```

2. httpplib.h

installation dans rep courant ou un rep dédié : exemple mkir \$HOME/cpp-libs

```
wget https://raw.githubusercontent.com/yhirose/cpp-httpplib/master/httpplib.h -O  
~/cpp-libs/httpplib.h
```

Remarque: l'option de repertoire -O est optionnelle ; on peut mettre -O . pour le repertoire courant

3. Exemple d'architecture et Modules à implementer

- l'enregistrement des routes => setupRoutes()

```
|  
|---- GET /status      → handleStatus() //etat du serveur OK, ERREUR etc  
|---- GET /mesures/temperature → handleGetMesuresTemperature() //lecture d'une  
ou liste de mesures de temperatures  
|---- GET /mesures/energie → handleGetMesuresEnergie() //lecture d'une ou liste de  
mesures d'énergie (Rappel :  $E \text{ (Wh)} = P \text{ (W)} * T \text{ (s)}$ )  
|---- POST /utilisateurs → handlePostMesure() //mise a jour des contacts  
administrateurs : nom, login, mdp, mail ,  
|---- PUT /seuils      → handleUpdateSeuils() // mise a jour de certains seuils alarme
```

- le démarrage du serveur (ouverture et ecoute sur le port par default 8080)
- la gestion des requêtes et réponses (analyse de la requete http et ecriture en json dans la reponse http)
- la logique métier derrière chaque route (code de chaque methode handle....)

4. Principe d'un serveur REST

Les principes d'une API REST (Representational State Transfer) reposent sur un ensemble de contraintes architecturales définies par Roy Fielding. Une API REST est un style d'architecture pour concevoir des services web légers, simples et scalables. Voici les principes fondamentaux :

1. Client-Serveur

- Le client (ex. une application mobile ou web) et le serveur (API) sont séparés.
- Cela permet une meilleure évolutivité et indépendance (chacun peut évoluer sans impacter l'autre tant que l'interface est respectée).

2. Stateless (sans état)

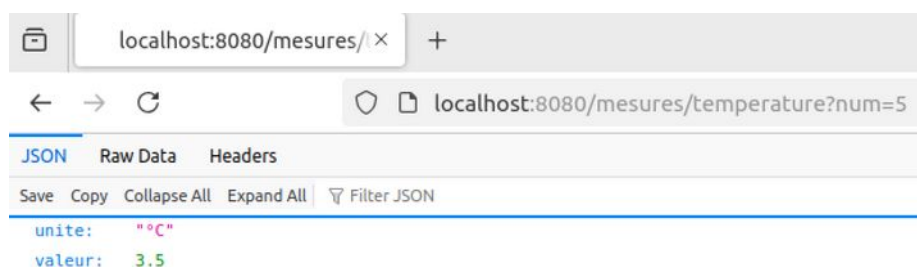
- Chaque requête envoyée au serveur doit contenir toutes les informations nécessaires (authentification, paramètres...).
- Le serveur **ne garde pas l'état de la session** du client entre deux requêtes.

3. Cacheable (cachable) Les réponses du serveur doivent indiquer si elles peuvent être mises en cache (via en-têtes HTTP par exemple).

- Cela améliore la performance et réduit la charge du serveur.
- **Interface uniforme (Uniform Interface)**
- Les ressources doivent être accessibles via des URL claires et stables.
- L'interaction se fait avec des méthodes standard HTTP :
 - **GET** → lire une ressource
 - **POST** → créer une ressource
 - **PUT/PATCH** → modifier une ressource
 - **DELETE** → supprimer une ressource
- Utilisation de **formats standard** (JSON, XML, etc.).

Coder les requetes issues d'un client web pour lire les mesures, ecrire les seuils d'alarme en base

Dans un premier temps , tout se fera par client non authentifié, les arguments des requetes seront passés dans l'URL Ex d'exécution ci -dessous.



5. Exemple de classe pour le serveur REST

fichiers : `sapc_rest_server.h` et `.cpp` (partiels donnés pour info)

Remarque : rappel des syntaxes liées aux méthodes “lambdas” (sans nom et définition préalable à leur utilisation) introduit depuis c++11

La capture permet d'utiliser des variables extérieures.

Syntaxe	Signification
[]	ne capture rien
[=]	capture tout par copie
[&]	capture tout par référence
[x]	capture x par copie
[&x]	capture x par référence

```

/** \class sapc_rest_server
 *
 * \author maylaender patrick
 * \version : 1.0
 * \date 02/26
 * \brief classe d'exploitation de requetes http */

class sapc_rest_server{

public:
    sapc_rest_server(int port=8080); //port 8080 par default
    /// setter et getter pour compost
    /// 1 seule valeur pour chaque paramètre
    void set_onetemperature(float temp){ mes_temperature = temp;}
    void set_onehumidite(float hum){ mes_humidite = hum;}

    float get_onetemperature(){ return mes_temperature;}
    float get_onehumidite(){ return mes_humidite;}

    ///vecteur (10 éléments dans un 1er temps...) de valeurs pour chaque paramètre
    void set_temperature(const std::vector<float>& temp);
    std::vector<float> get_temperature() const { return val_temp; }

    ///batterie(s)
    ///setter et Getter
    void set_courant(const std::vector<float>& curr);
    std::vector<float> get_courant() const { return val_courant; }

    ///lance le serveur REST sur port par default
    void start();

private:
    ///analyse des routes (methodes) appeles selon argument dans requete
    void setupRoutes();
    ///TO DO : autres methodes metier utiles

    void handleGetMesuresTemperature(const httplib::Request& req, httplib::Response& res);
    void handleGetMesuresPuissance(const httplib::Request& req, httplib::Response& res);

    void handleGetStatus(const httplib::Request& req, httplib::Response& res);
    void handlePostSeuils(const httplib::Request& req, httplib::Response& res);

    httplib::Server server;
    int port;

    //PM 24/2/26
    std::vector<float> val_courant ; //mesures initialisées dans constructeur
    std::vector<float> val_temp ;
    float mes_temperature, mes_humidite;

};

```

```

sapc_rest_server.cpp
#include "sapc_rest_server.h"
//declare espace de noms lié à la lib inclue dans le rep systeme /usr/include/nlohmann
using json = nlohmann::json;

/** \fn void sapc_rest_server(int newport):val_courant(10, 0.0f), val_temp(10,0.0f)
\brief constructeur avec valeurs par defaut 0.0 (float) pour chaque element des vecteurs de mesures
**/
sapc_rest_server::sapc_rest_server(int newport):val_courant(10, 0.0f), val_temp(10,0.0f)
{
    this->port = newport;

    this->mes_temperature = 0.0;
    this->mes_humidite = 0.0;

}
//methode de start du serveur Rest
void sapc_rest_server::start()
{
    setupRoutes(); ///appel le handle de routes
    server.listen("0.0.0.0", this->port); //ecoute sur le port par defaut de la socket serveur
}

// Setter pour vecteur de température
void sapc_rest_server::set_temperature(const std::vector<float>& temp)
{
    for (size_t i = 0; i < val_temp.size() && i < temp.size(); ++i) {
        val_temp[i] = temp[i];
    }
}

//setter pour vecteur de puissance TO DO
//setter pour vecteur d'humidité TO DO

/*****
** \fn void setupRoutes()
\brief methode d'aiguillage vers les methodes metiers selon la requete GET,POST....
\param[in]   neant
\param[out]  neant
\return      neant
*****/
void sapc_rest_server::setupRoutes(){
    //TO DO : Ajouter autant de routes que de methodes necessaires : mesures energetique
    (Wh), Temperature, humidite, et leurs paramètres éventuels : dates concernées

    //capture, par cette méthode "lambda", de l'objet global [this] pour appeler toute
    methode implémentée dans la classe instanciée
    //lecture de la temperature dans le compost, lue par le systeme
    server.Get("/mesures/temperature",[this](const http::Request& req,http::Response& res)
    {
        handleGetMesuresTemperature(req, res);
    });
}

```

```

//lecture de la puissance prise sur batterie(s) par le systeme
server.Get("/mesures/puissance",[this](const httpplib::Request& req,httpplib::Response& res)
{
    handleGetMesuresPuissance(req, res);
});
//mise à jour seuils alarmes
server.Post("/seuil_alarmes",[this](const httpplib::Request& req,httpplib::Response& res)
{
    handlePostSeuils(req, res);
});
//etat du serveur
server.Get("/status", [this](const httpplib::Request& req,httpplib::Response& res)
{
    handleGetStatus(req,res);
});
//etc... }
/*****
/** \fn void handleGetMesuresTemperature(const httpplib::Request& req, httpplib::Response& res)
    \brief methode de lecture requete http et reponse en REST : les N dernieres mesures(T°)
    \param[in] const httpplib::Request& req
    \param[out] httpplib::Response& res
    \return neant
*****/
void sapc_rest_server::handleGetMesuresTemperature(const httpplib::Request& req,
httpplib::Response& res)
{
//TO DO (PM 26/2/26) : remplacer par vecteurs de N elements
    std::vector<float> temperature(10) ; //TO DO un new (vs 10) pour les N (vs 10) dernieres
mesures

    float value = 0.0f; // SI 1 seule valeur demandée
//appels des getters CF .h
    temperature = this -> get_temperature(); //recupere les 10 valeurs de temperature

    if (req.has_param("num")) { //suppose parametre num pour numero mesure dans la requete URL
        if (!temperature.empty()) { //au moins une mesure dans le vecteur

            int n = 0; //par default
            try {
                n = std::stoi(req.get_param_value("num"));
                value = temperature.at(n); // nieme mesure

                nlohmann::json j; //objet json pour reponse

                j= {{"valeur",value},{"unite","°C"}};
                //envoi en JSON dans le cors de la reponse http
                res.set_content(j.dump(), "application/json");
            } catch (...) {
                res.status = 400; //erreur
                nlohmann::json j;
                j["error"] = "Type inconnu : Paramètre num invalide ";
            }
        }
    }
}

```

```

        //envoi en JSON dans le cors de la reponse http
        res.set_content(j.dump(), "application/json");

        return;
    }
}
else {
    res.status = 404;
    nlohmann::json j;
    j["error"] = "Pas de mesures de temperatures disponibles ";
    res.set_content(j.dump(), "application/json");
    return;
}
}
.... //toutes les mesures A Faire dans un tableau json::array()
} //fin methode

```

//fichier main_rest_sapc.cpp

//Exemple du main()

#include "sapc_mcp3008.h"

#include "sapc_ina226.h"

#include "sapc_rest_server.h"

using namespace std;

int main()

{

//beta test sans base de données

std::vector<float> courant(10, 1.0f); // initialise 10 mesures reelles à 0

std::vector<float> temperature(10, 2.0f); // initialise 10 mesures reelles à 2.0

sapc_rest_server server(8080);

server.start(); //demarre serveur REST

temperature.at(5) = 3.5 ; //beta test 3.5 V est ecrit dans 6eme element voir doc capteur pour loi d'echelle °C / Volt et %humidite / Volt

#ifdef INA

//Partie capteur ina226

TO DO en phase d'integration

int i = 0;

while(i < 10){

//lecture en base ensuite

//TO DO courant.at(i) = lecture valeur

i++;

}


```

#endif

#ifdef MCP
//ifdef _SAPC_MCP3008
//PARTIE MCP

int j = 0;
while (j < 10)
{
//TO DO en phase d'integration
temperature.at(j) = appel methode de lecture CF doc capteur

std::cout << "CH0 = " << temperature.at(j) << "°C" << std::endl;

std::this_thread::sleep_for(
    std::chrono::milliseconds(5000));
    j++;
}

#endif

server.set_temperature(temperature); //copies les 10 valeurs mesurees dans le vecteur attribut //
TEST ONLY
server.set_courant(courant);

//APPEL des methodes de la classe BDD
//detection des alertes
....
return 0;
}

```

CAPTEUR INA226 (B.Sahan)

6. Gestion du capteur ina226

L'interface associée est dans `sapc_ina226.h`

Le code des methodes est à poursuivre dans le `.cpp` !!

Remarque sur la mise en oeuvre du capteur lui-même :

Il est fondamental de brancher les broches **IN+** et **IN-** en **SERIE**, sur la ligne issue de la borne +12v de la (des) batterie(s) LiPO.

Remarque : Dans un premier temps vous prendrez une alimentation stabilisée **SORTIE COURANT et réglée à Max 1A. Voir avec M.Wozniak**

Voir le schéma pdf si besoin. Bientot disponible TO DO PM 2/3/26

Les registres importants à configurer et lire sont tous résumés dans la figure ci-dessous :

All 16-bit device registers are two 8-bit bytes via the I²C interface.

La

Table 7-1. Register Set Summary

POINTER ADDRESS HEX	REGISTER NAME	FUNCTION	POWER-ON RESET		TYPE ⁽¹⁾
			BINARY	HEX	
00h	Configuration Register	All-register reset, shunt voltage and bus voltage ADC conversion times and averaging, operating mode.	01000001 00100111	4127	R/ W
01h	Shunt Voltage Register	Shunt voltage measurement data.	00000000 00000000	0000	R
02h	Bus Voltage Register	Bus voltage measurement data.	00000000 00000000	0000	R
03h	Power Register ⁽²⁾	Contains the value of the calculated power being delivered to the load.	00000000 00000000	0000	R
04h	Current Register ⁽²⁾	Contains the value of the calculated current flowing through the shunt resistor.	00000000 00000000	0000	R
05h	Calibration Register	Sets full-scale range and LSB of current and power measurements. Overall system calibration.	00000000 00000000	0000	R/ W
06h	Mask/Enable Register	Alert configuration and Conversion Ready flag.	00000000 00000000	0000	R/ W
07h	Alert Limit Register	Contains the limit value to compare to the selected Alert function.	00000000 00000000	0000	R/ W
FEh	Manufacturer ID Register	Contains unique manufacturer identification number.	0101010001001001	5449	R
FFh	Die ID Register	Contains unique die identification number.	0010001001100000	2260	R

(1) Type: **R** = Read-Only, **R/ W** = Read/Write.

(2) The Current Register (04h) and Power Register (03h) default to '0' because the Calibration register defaults to '0', yielding zero current and power values until the Calibration register is programmed.

méthode proposée dans l'interface `ina226.h` possède le prototype :

`bool INA226::calibrate(double shuntOhm, double maxCurrent);`

Son rôle est d'écrire dans le registre *Calibration register* (d'adresse 05h) la valeur de la résistance de shunt 0.1ohms (mesure effectuée par P.Maylaender), et un courant max de 1 A donc 1.0 mais à reconfirmer sur TP à faire avec conseils de mise en oeuvre par M.Wozniak)

Le code de cette méthode fait globalement ce qui est décrit dans la doc `ina226.pdf` (page 15) et dont on rappelle ci-dessous le rôle :

The Calibration Register is calculated based on Equation 1. This equation includes the term Current_LSB, which is the programmed value for the LSB for the Current Register (04h). The user uses this value to convert the value in the Current Register (04h) to the actual current in amperes. The highest resolution for the Current Register (04h) can be obtained by using the smallest allowable Current_LSB based on the maximum expected current as shown in Equation 2. While this value yields the highest resolution, selecting a value for the Current_LSB to the nearest round number above this value is common to simplify the conversion of the Current Register (04h) and Power Register (03h) to amperes and watts respectively. The R_{SHUNT} term is the value of the external shunt used to develop the differential voltage across the input pins.

$$CAL = \frac{0.00512}{Current_LSB \times R_{SHUNT}} \quad (1)$$

where

- 0.00512 is an internal fixed value used to verify scaling is maintained properly

$$Current_LSB = \frac{Maximum\ Expected\ Current}{2^{15}} \quad (2)$$

Exemple d'appel dans le *main()* du fichier *main_rest_sapc.cpp*

Remarque : Le code est opérationnel en librairie (libina.a) mais c'est votre travail de proposer un code pour chaque méthode proposée dans le .h

CAPTEUR MCP3008 (K.Faille)

7. Gestion du capteur MCP3008

L'interface proposée comme guide est dans `sapc_mcp3008.h`

Le code des methodes est à poursuivre dans le .cpp !!

Principes généraux :

Une méthode de la classe, devra ouvrir le device en retournant un descripteur de fichier (*file descriptor en anglais*, entier pris dans la table des descripteurs de tout processus unix), et fermer (le retirer de la table) en fin de programme !

Ce descripteur sera utilisé ensuite pour toute les opérations de type lecture / écriture : `read()`, `write()` Cf (*man read* et *man 2 write* pour doc aide en ligne et `ioctl()` (fonction acces direct au driver en passant un numéro de fonction en argument)

Cf man `ioctl`

Le principe important pourra etre d'utiliser une structure type

```
struct spi_ioc_transfer tr {};
```

```
tr.tx_buf = (unsigned long)tx; //buffer emission pointe sur tx
```

```
tr.rx_buf = (unsigned long)rx; //buffer reception pointe sur rx
```

Chaque transfert sur le bus SPI repose sur des commandes en 3 octets

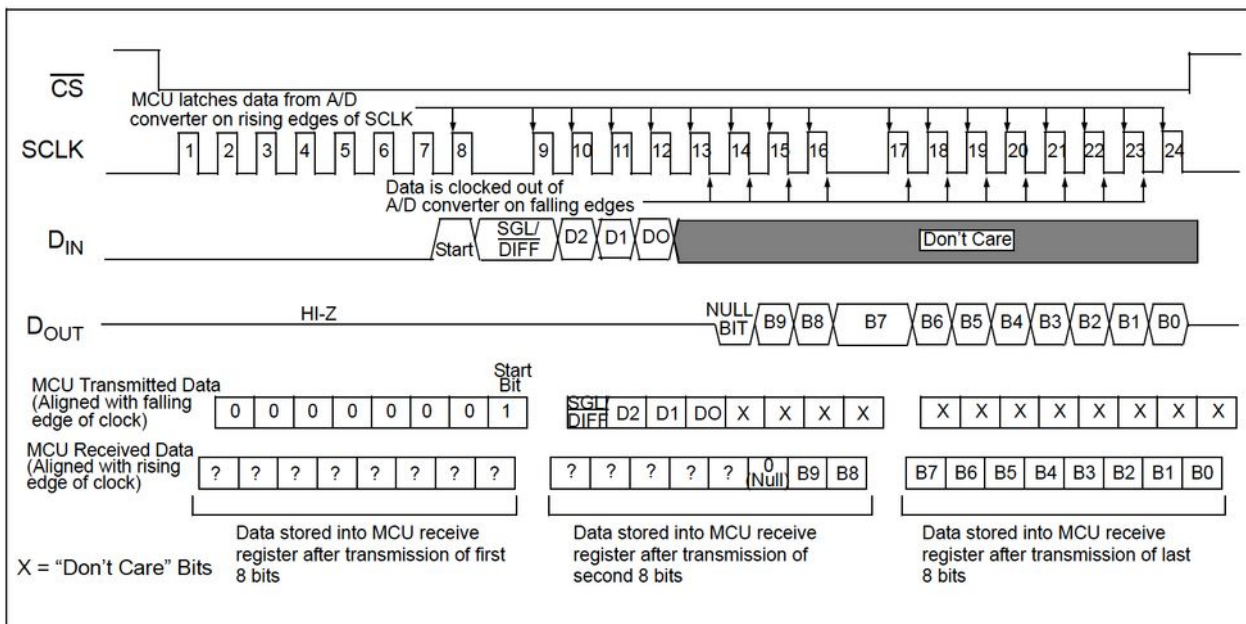


FIGURE 6-1: SPI Communication with the MCP3004/3008 using 8-bit segments (Mode 0,0: SCLK idles low).

Les constantes utiles en programmation sont toutes déclarées dans les headers système

```
#include <linux/spi/spidev.h>
```

```
#include <sys/ioctl.h>
```

La methode conseillée (ou equivalente) bool MCP3008::openDevice() devra faire au minimum les 3 opération suivantes :

```
ioctl(fd, SPI_IOC_WR_MODE, &mode);  
ioctl(fd, SPI_IOC_WR_BITS_PER_WORD, &bits);  
ioctl(fd, SPI_IOC_WR_MAX_SPEED_HZ, &spiSpeed);
```

où fd est le file descriptor, le 2ème argument est la fonction demandée au travers de l'accès ioctl, le 3eme sera la variable contenant la valeur du paramètre à régler. (mode du bus SPI, nombre de bits de données / octet et vitesse du bus)

Remarque sur la mise en oeuvre du capteur lui-même :

Le capteur délivre une tension pour l'humidité et la température entre 0 et 10v. Cf doc du Bbtechnik
Ce capteur est alimenté en 0 – 12v .

Une mise en forme electronique est faite sur la carte proto shield montée sur le pi3.

Voir le schéma pdf si besoin. Lien à faire TO DO PM 2/3/26

Une adaptation entre 0-3,3V est donc faite. Il faudra en tenir compte sur les calculs d'échelle sur les valeurs renvoyées par le CNA. Ex : nombre n = 0 <=> tension 0 : -30° C (peu probable...)

Max 70°C (45°C probable dans un composteur) pour 3,3v et donc une loi linéaire sera à determiner (et coder) pour avoir $T^{\circ}C = f(n)$

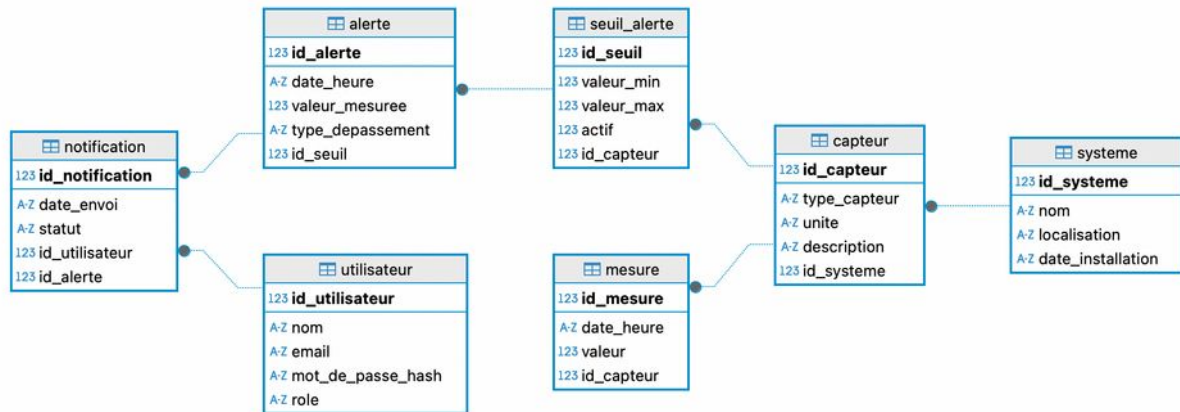
Cf ina226.h `float calcul_parametre(int val_issue_cna, float Vmin, float Vmax);`

Remarque1 : Dans un premier temps, vous lirez sur un simple potentiomètre alimenté en 3.3V sur un des canaux du MCP : canal CH0 par exemple. Les 2,3 premiers sont cablés sur la carte protoshield.

Remarque2 : Le code est opérationnel en librairie (libmcp.a) mais c'est votre travail de proposer un code pour chaque méthode proposée dans le .h

8. Base de données (G.Malherbe)

Le MCD complet (trop peut être) pourrait être le suivant :



Il a été généré à partir du fichier .sql constituant la base de données. Extrait pour information ci-dessous :

Remarque : le modèle ci-dessus peut se simplifier : la table système n'est pas utile pour nous car un seul "système" compositeur.

```
-- =====
-- TABLE CAPTEUR
-- =====
```

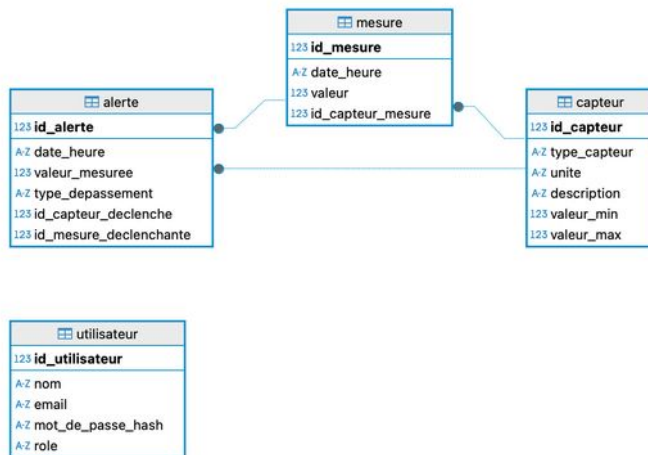
```
CREATE TABLE capteur (
  id_capteur INTEGER PRIMARY KEY AUTOINCREMENT,
  type_capteur TEXT NOT NULL,
  unite TEXT,
  description TEXT,
  id_systeme INTEGER NOT NULL,
  FOREIGN KEY (id_systeme)
    REFERENCES systeme(id_systeme)
    ON DELETE CASCADE
);
```

```
-- =====
-- TABLE MESURE
-- =====
```

```
CREATE TABLE mesure (
  id_mesure INTEGER PRIMARY KEY AUTOINCREMENT,
  date_heure DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  valeur REAL NOT NULL,
  id_capteur INTEGER NOT NULL,
  FOREIGN KEY (id_capteur)
    REFERENCES capteur(id_capteur)
    ON DELETE CASCADE
);...
```

Ce fichier est ensuite “lisible” par des logiciels type MysqlWorkbench, DBeaver, DBrowser par exemple. Faire un import de votre fichier.sql puis faire selon l’outil des options comme reverse engineer, ER diagram...

Modele plus simple



9. Installation sqlite3 et developpement de la classe en C++

L’interface à développer devra reposer sur sqlite3 directement donc un `#include <sqlite3.h>` dans votre(vos) header/ source(s) .h / .cpp ET l’installation des packages

```
sudo apt update
sudo apt install sqlite3 libsqlite3-dev
```

Vérification par : `sqlite3 --version`
Lancement en console par `sqlite3`

La compilation fera intervenir la librairie sqlite3 : *CF paragraphe 12*
ex : `g++ main.cpp -lsqlite3 -o test_sqlite`

Remarque pratique : Dans un premier temps vous pouvez mettre au point des codes en C pour dialoguer avec votre base créée en sql.

Quand cela fonctionnera pour des fonctions simples permettant d’exécuter vos requetes sql vous pourrez migrer en C++. Il “suffira” d’encapsuler votre code C dans une classe C++. Exemple “minimal” en paragraphe 10.

Remarque : Sqlite ne gere pas l’authentification en base. Il faut gerer si on le souhaite, les mots de passe et faire une fonction de hachage. Cf 10

Il faut pour cela installer la librairie ssl-dev par : `sudo apt install libssl-dev`

10. Exemple de classe

sapc_bdd.h

```
#include <sqlite3.h>
#include <string.h>

#include <openssl/sha.h>
#include <iostream>
#include <sstream>
#include <iomanip>
class sapc_bdd
{
public :
    ///init classe metier Base de données

    sapc_bdd(std::string file);
    //creation (ajout) d'un utilisateur dans la bdd avec son nom, login, mot de passe crypté, mail et son role
    bool createUser(std::string nom, std::string login, std::string password, std::string email, std::string role);
    //verif si utilisateur connu dans la base
    bool login(std::string nom, std::string login, std::string password);

    void fin();
private:

    sqlite3 *db; //pointeur instance bdd
    std::string sha256(const std::string& str); //methode interne a la librairie ssl permettant de crypter le mot de passe

};
```

source : sapc_bdd.cpp

```
#include "sapc_bdd.h"

//constructeur (nom fichier de base en argument)
sapc_bdd::sapc_bdd(std::string file)
{
    char* errMsg = 0;
    int res = 0;
    std::string sql ;
    ///
    /// Connexion à la base de donnée
    if (sqlite3_open(file.c_str(), &db)) {
        std::cout << "Erreur ouverture DB\n";
    }
}
```



```

// Activer les foreign keys
res = sqlite3_exec(db, "PRAGMA foreign_keys = ON;", nullptr, nullptr, nullptr);
if(res != SQLITE_OK) {
std::cerr << "Erreur lors de l'activation des foreign keys : " << sqlite3_errmsg(db) << std::endl;
} else {
std::cout << "Base ouverte et foreign_keys activées." << std::endl;
}

//creation des tables
//utilisateur
sql = R"(CREATE TABLE IF NOT EXISTS utilisateur (
id_utilisateur INTEGER PRIMARY KEY AUTOINCREMENT,
nom TEXT NOT NULL,
login TEXT NOT NULL,
mot_de_passe_hash TEXT NOT NULL,
email TEXT NOT NULL UNIQUE,
role TEXT CHECK(role IN ('ADMIN','USER')) DEFAULT 'USER'
);)";

if (sqlite3_exec(db, sql.c_str(), 0, 0, &errMsg) != SQLITE_OK) {
std::cout << "Erreur SQL: " << errMsg << "\n";
sqlite3_free(errMsg);
}

//capteur TO DO
.....
//mesures , alertes ...TO DO

//fin constructeur : creation normale objet
}

//methode de hachage pour mot de passe car non gere par sqlite directement
std::string sapc_bdd::sha256(const std::string& str)
{
    unsigned char hash[SHA256_DIGEST_LENGTH];

    //crypte avec algo SHA 256
    SHA256((unsigned char*)str.c_str(), str.size(), hash);

    std::stringstream ss;

    for(int i=0;i<SHA256_DIGEST_LENGTH;i++)
    {
        //convertit en hexadecimal la chaine cryptee
        ss << std::hex << std::setw(2)
        << std::setfill('0') << (int)hash[i];
    }
    return ss.str();
}

```

///creation utilisateur

bool sapc_bdd::createUser(std::string nom, std::string login, std::string **password**, std::string email, std::string role)

```
{
    sqlite3_stmt* stmt;

    std::string sql =
        "INSERT INTO utilisateur(nom, login,mot_de_passe_hash, email, role) VALUES(?,?,?,?,?)";

    sqlite3_prepare_v2(db, sql.c_str(), -1, &stmt, NULL);

    //hache le password avant insertion !
    std::string hash = sha256(password);

    sqlite3_bind_text(stmt,1,nom.c_str(),-1,SQLITE_TRANSIENT);
    sqlite3_bind_text(stmt,2,login.c_str(),-1,SQLITE_TRANSIENT);
    sqlite3_bind_text(stmt,3,hash.c_str(),-1,SQLITE_TRANSIENT);
    sqlite3_bind_text(stmt,4,email.c_str(),-1,SQLITE_TRANSIENT);
    sqlite3_bind_text(stmt,5,role.c_str(),-1,SQLITE_TRANSIENT);

    int rc = sqlite3_step(stmt);

    sqlite3_finalize(stmt);

    return rc == SQLITE_DONE;
}
```

//PM 15/3/26

//test si utilisateur connu sur base de son nom, login et mdp haché ; voir si email et role à ajouter

bool sapc_bdd::login(std::string nom, std::string login, std::string password)

```
{
    sqlite3_stmt* stmt;
    std::string sql =
        "SELECT id_utilisateur FROM utilisateur WHERE nom=? AND login=? AND mot_de_passe_hash=?";

    sqlite3_prepare_v2(db, sql.c_str(), -1, &stmt, NULL);

    //hache le password avant comparaison !
    std::string hash = sha256(password);

    sqlite3_bind_text(stmt,1,nom.c_str(),-1,SQLITE_TRANSIENT);
    sqlite3_bind_text(stmt,2,login.c_str(),-1,SQLITE_TRANSIENT);
    sqlite3_bind_text(stmt,3,hash.c_str(),-1,SQLITE_TRANSIENT);

    int rc = sqlite3_step(stmt);

    sqlite3_finalize(stmt);

    return rc == SQLITE_ROW;
}
```

```

void sapc_bdd::fin()
{
    //insere temps de fin si arret prog ou scenario
    sqlite3_close(db);
    exit(0);
}

//main exemple
#include <sqlite3.h>
#include <iostream>
#include "sapc_bdd.h"

int main(int argc, char **argv) {

    sapc_bdd BDD(argv[1]) ; //creer la base avec nom en argument

    if(!BDD.createUser("PROF","admin","1234","test@outlook.fr", "ADMIN"))
    {
        std::cerr<<"Erreur creation user "<< std::endl;
    }
    else
    {
        std::cout<<"creation user admin --> ok" <<std::endl;
    }
    //verification si utilisateur PROF existe
    if(!BDD.login("PROF","admin", "1234"))
    {
        std::cerr<<"Mauvais login ou mot de passe"<< std::endl;
    }
    else
    {
        std::cout<<"Authentification PROF admin --> ok" <<std::endl;
        BDD.fin(); //ferme connexion socket base
        return 0;
    }
}

```

11. Compilation minimale :

```
g++ -Wall -o sapc_bdd main_test_bdd.cpp sapc_bdd.cpp -lsqlite3 -lcrypto
```

ou en utilisant le Makefile

make -f Makefile2 libbdd puis make all

12. Test

```

pm@pm-virtual-machine:~/Documents/TS2/SAPC/v2$ ./sapc_bdd sapc_bdd.db
Base ouverte et foreign_keys activées.
creation user admin --> ok
Authentification PROF admin --> ok

```